

KEY FORMULA

Symbols

Symbol(s)	Meaning	Value / Notes
Architecture (fixed per model)		
L, L_{moe}	transformer layers; layers carrying MoE	43/43, 61/61, 61/58
n_{csa}, n_{hca}	CSA / HCA layer counts	from compress_ratios
e, ι	bytes per KV entry; per CSA indexer key	576, 68
m, m'	CSA / HCA compression ratios	4, 128
n_{vin}, k_{top}	sliding window; CSA sparse top-k	128; 512/1024
n_e, k	routed experts per layer; activated per token	256/384; 6/8
w_e, W_0	parameters per expert; non-expert weight bytes	
b_e, b	bytes per expert param; per stored KV element	0.53125 (MXFP4) / 1.0
P_{act}, c_{max}	activated params per token; context ceiling	
d_{kv}, d_r	MLA latent / decoupled-RoPE dims (dense case)	512, 64
Workload, persona i		
s_i, c_i, o_i	seat share; context tokens; output tokens	
r_i, h_i	requests per active user per day; prefix-cache hit rate	
λ_i, C_i	peak arrival rate; concurrent sessions	
Deployment		

Symbol(s)	Meaning	Value / Notes
N, a, φ	seats; daily adoption; peak-hour concentration	
ℓ , ψ	planned load factor; idle-KV multiplier	0.70, 1.0
τ , T_0	TPOT SLO (s/token); TTFT budget (s)	0.040, 2.0
η_m , η_β , η_{mfu}	usable HBM; achievable bandwidth; prefill MFU	0.85, 0.70, 0.40
σ , R, v	expert routing skew; redundancy; GPUs per node	1.15, 2, 8
Hardware		
H, β , F	HBM bytes; HBM bandwidth; dense FP8 FLOP/s	

Formula

Stage 0 — Model constants (deployment-independent)

$\kappa = n_{\text{csa}} \frac{e + \iota}{m} + n_{\text{hca}} \frac{e}{m'}$	KV bytes <i>stored</i> per context token
$A = e (n_{\text{csa}} k_{\text{top}} + L n_{\text{win}})$	KV bytes <i>read</i> per step, context-free
$B = n_{\text{csa}} \frac{\iota}{m} + n_{\text{hca}} \frac{e}{m'}$	KV bytes <i>read</i> per step, per context token
$\rho(c) = A + Bc$	KV read per sequence per decode step
$W = L_{\text{moe}} (n_e + 1) w_e b_e + W_0$	total weight bytes

Stage 1 — Demand to concurrency (Little’s Law)

$$\lambda_i = \frac{N s_i a r_i \varphi}{3600}$$

$$\gamma_i = \frac{a \varphi}{3600 \ell} s_i r_i (T_0 + o_i \tau), \quad \gamma = \sum_i \gamma_i, \quad C_i = \gamma_i N, \quad C = \gamma N$$

Stage 2 — Capacity

$$M = W + \psi \kappa \sum_i C_i c_i = W + \psi \kappa \bar{c} \gamma N, \quad G_{\text{cap}} = \frac{M}{H \eta_m}$$

Stage 3 — Decode bandwidth

$$W_{\text{eff}}(C) = L_{\text{moe}} \left(n_e \min \left[1, \sigma \left(1 - \left(1 - \frac{k}{n_e} \right)^C \right) \right] + 1 \right) w_e b_e + W_0$$

$$S(C) = W_{\text{eff}}(C) + A C + B \sum_i C_i c_i, \quad G_{\text{bw}} = \frac{S(C)}{\beta \eta_b \tau}$$

Stage 3b — Prefill

$$\Pi = \sum_i \lambda_i c_i (1 - h_i) \quad \text{uncached prompt tokens/s at peak}$$

$$G_{\text{pf}} = \frac{2 \Pi P_{\text{act}}}{F \eta_{\text{mfu}}} \quad \text{prefill GPUs}$$

Stage 4 — Feasibility and fleet

$$\text{Infeasible if } \max_i c_i > c_{\text{max}}$$

$$\mathcal{S}(g) = \begin{cases} \min\{x \in \{1, 2, 4, \nu\} : x \geq g\}, & g \leq \nu \\ \nu \lceil g/\nu \rceil, & g > \nu \end{cases} \quad (\text{snap to a valid TP/EP degree})$$

$$G = R[\mathcal{S}(\max(G_{\text{cap}}, G_{\text{bw}})) + \mathcal{S}(G_{\text{pf}})]$$

Derived invariants

$$\Lambda = \frac{W}{\kappa} \quad \text{live-context tokens whose KV equals } W$$

$$\Phi = \frac{B}{\kappa} \quad \text{re-read fraction (dense} = 1)$$

$$C_{\text{sat}} = \frac{\ln(1 - \varepsilon)}{\ln(1 - k/n_e)} \quad \text{batch at which } W_{\text{eff}} \rightarrow \varepsilon W$$

$$N^* = \frac{\Lambda}{\psi \gamma \bar{c}} \quad \text{headcount where KV memory} = W$$

The latex formulas are rendered by google gemini. Not produced by it.

UNCERTAINTY REGISTER

What this register is for:

Explains **which numbers are derived and verified, which are absorbed into the tool as parameters, and which are still guesses.**

Key disclaimer:

Model constants were derived from architecture and confirmed field-by-field against config.json; workload inputs were invented. Those deserve very different levels of trust and the fleet number alone doesn't distinguish them.

Broadly the rows fall into three classes:

Architectural facts (U1) are binary and resolved by a procurement decision, not by more analysis.
Derived model constants (U2, U3, U5, U6) are narrow and verified; collectively they move the fleet by roughly one GPU.

Workload and efficiency assumptions (U4, U7–U11) are wide, unmeasured, and can only be closed by a pilot.

#	Source	Affects	Magnitude	Modelled?	Character
U1	FP4 hardware path	W	+84% Flash, +86% Pro (+727 GB on Pro)	Yes — HW[].fp4_native	Binary. A procurement decision, not an estimate. Dominates all other rows combined.
U8	GPU FLOPS figures unverified	G_{pf}	Unknown; prefill is ~50% of the Hopper fleet	No	Values in HW[] are from memory. Verify against vendor datasheets before spending.
U9	Prefix-cache hit rates assumed (0.30 / 0.85 / 0.20)	G_{pf}	High leverage — sets Π directly	Input only	Measurable in a pilot; currently pure assumption.

#	Source	Affects	Magnitude	Modelled?	Character
U4	Expert routing skew	G_{bw}	+10–20%	Yes — $\sigma = 1.15$	Absorbed, but σ itself is unvalidated. Stragglers set step time under EP.
U7	Arrival burstiness	C	Tail exceeds γN	Partly — $\ell = 0.70$	φ and ℓ are crude proxies for a real queueing model.
U10	Prefill MFU = 0.40	G_{pf}	$\pm 25\%$ plausible	Yes — constant	Absorbed as a fixed parameter; not validated on this architecture.
U11	Interconnect / EP network limits	G_{bw}	May bind at ≥ 64 GPUs	No	Tech report gives a 6,144 FLOPs/byte balance point for Pro; unmodelled.
U3	MTP module loaded	W	+3.5 / +13.8 / +11.5 GB	Yes — <code>--mtp</code>	Optional. Zero κ impact (compress_ratio: 0 confirmed for both MTP blocks).
U5	V4-Flash config provenance	Flash constants	Low	No	Third-party FlagOS mirror; every field cross-checks against the tech report.
U2	Indexer keys persisted?	κ	–11.5% / –11.4% on κ ; 0 GPUs on output	Yes — <code>--band</code>	Diagnosed: removing the indexer term reproduces the paper's figures to <0.5% on both models. Immaterial to the fleet.
U6	Embedding precision	W	± 0.5 / ± 0.9 GB	No	Negligible.

USING sizing_v2.py :

```
python3 sizing_v2.py          # full report, default assumptions
python3 sizing_v2.py --seats 12000 # your headcount
python3 sizing_v2.py --sweep    # scaling curve 500→50,000 seats; B200 only, ignores --seats
python3 sizing_v2.py --band lo  # optimistic KV accounting
python3 sizing_v2.py --mtp      # include MTP for speculative decoding
```

The CLI covers four common overrides. Everything else — SLOs, efficiency factors, redundancy, the workload mix, and hardware — is edited in the DEPLOY, MIX, and HW blocks at the foot of the file. Requires Python 3.8+, standard library only. Model constants self-validate on import and raise AssertionError if edited inconsistently.

SOURCE CODE

```
#!/usr/bin/env python3
"""
GPU sizing for DeepSeek-V4-Flash / V4-Pro / R1-671B.    v2

    demand --> concurrency --> {capacity, bandwidth, prefill} --> topology

USAGE
-----
python3 sizing_v2.py                # full report, default assumptions
python3 sizing_v2.py --seats 12000   # your headcount
python3 sizing_v2.py --sweep        # scaling curve 500→50,000 seats; B200 only, ignores --seats
python3 sizing_v2.py --band lo      # optimistic KV accounting
python3 sizing_v2.py --mtp          # include MTP for speculative decoding

Edit DEPLOY / MIX / HW at the bottom for anything else. Every assumption is
a named constant; nothing is buried in a function.

NOT MODELLED (know these before you trust the output)
-----
* queueing dynamics: phi is a crude peak proxy, not a tail model
* prefix-cache hit rates are assumed, not measured
* network/interconnect limits in expert parallelism
* disaggregated prefill/decode scheduling overhead
* cold-start, checkpoint load, model-swap time
* FLOPS figures in HW[] are from memory -- VERIFY against vendor datasheets
"""
import argparse, math

GB, MB, TB = 1e9, 1e6, 1e12

# =====
# MODEL CONSTANTS
# Derived from architecture; validated against reported totals to <0.5% and
# confirmed field-by-field against config.json for all three models.
# e   = 576 B   compressed KV entry (64 RoPE dims BF16 + 448 dims FP8)
# iota= 68 B   CSA indexer key (MXFP4, 128 dims + E8M0 scale)
```

```

# =====
E_ENTRY, IOTA, M_CSA, M_HCA, N_WIN = 576, 68, 4, 128, 128
FP4 = 0.5 + 1/32          # MXFP4 incl. E8M0 scale, confirmed scale_fmt=ue8m0
FP8 = 1.0

class Model:
    def __init__(self, name, *, n_csa, n_hca, topk, L, d, d_ff, n_e, k,
                  W_attn, W_emb, W_dense_ffn=0.0, c_max, dense_mla=False,
                  L_moe=None, kv_dense=None, P_act):
        self.name, self.L, self.n_e, self.k = name, L, n_e, k
        self.L_moe = L_moe if L_moe is not None else L
        self.c_max, self.P_act = c_max, P_act
        self.w_e = 3 * d * d_ff          # params per expert
        self.n_experts_per_layer = n_e + 1  # routed + shared
        self.W_attn, self.W_emb, self.W_dense_ffn = W_attn, W_emb, W_dense_ffn
        self.dense_mla = dense_mla

        if dense_mla:                    # R1: full MLA, every layer
            self.kappa_hi = self.kappa_lo = L * kv_dense * 2
            self.A, self.B = 0.0, L * kv_dense * 2
        else:
            self.kappa_hi = n_csa*(E_ENTRY+IOTA)/M_CSA + n_hca*E_ENTRY/M_HCA
            self.kappa_lo = n_csa*E_ENTRY/M_CSA + n_hca*E_ENTRY/M_HCA
            self.A = E_ENTRY * (n_csa*topk + L*N_WIN)
            self.B = n_csa*IOTA/M_CSA + n_hca*E_ENTRY/M_HCA

# ---- weights depend on the precision path: uncertainty U1 -----
def W(self, precision="blackwell", mtp=False):
    expert_params = self.L_moe * self.n_experts_per_layer * self.w_e
    b = FP4 if (precision == "blackwell" and not self.dense_mla) else FP8
    w = expert_params*b + self.W_attn*FP8 + self.W_dense_ffn*FP8 + self.W_emb*2
    if mtp:                             # one extra block, MoE-shaped
        w += self.n_experts_per_layer*self.w_e*b + self.W_attn/self.L*FP8
    return w

def kappa(self, band="hi"):
    return self.kappa_hi if band == "hi" else self.kappa_lo

def rho(self, c):                      # KV read per decode step
    return self.A + self.B * c

def W_eff(self, C, precision, mtp, skew=1.0):
    """Weight bytes touched at batch C. skew>1 models non-uniform routing."""
    frac = min(1.0, skew * (1 - (1 - self.k/self.n_e)**C))
    b = FP4 if (precision == "blackwell" and not self.dense_mla) else FP8
    moe = self.L_moe * (self.n_e*frac + 1) * self.w_e * b
    return moe + self.W_attn*FP8 + self.W_dense_ffn*FP8 + self.W_emb*2

# ---- deployment-independent invariants -----
def Lam(self, precision="blackwell", band="hi"):    # tokens where KV == W
    return self.W(precision) / self.kappa(band)

```

```

def Phi(self, band="hi"):
    # re-read fraction
    return self.B / self.kappa(band)
def C_sat(self, eps=0.95):
    return math.log(1-eps) / math.log(1 - self.k/self.n_e)

FLASH = Model("V4-Flash", n_csa=21, n_hca=20, topk=512, L=43, d=4096, d_ff=2048,
               n_e=256, k=6, W_attn=4.83e9, W_emb=2*129280*4096,
               c_max=1_048_576, P_act=13.6e9)
PRO    = Model("V4-Pro",   n_csa=30, n_hca=31, topk=1024, L=61, d=7168, d_ff=3072,
               n_e=384, k=6, W_attn=19.2e9, W_emb=2*129280*7168,
               c_max=1_048_576, P_act=49.3e9)
R1     = Model("R1-671B",  n_csa=0, n_hca=0, topk=0, L=61, d=7168, d_ff=2048,
               n_e=256, k=8, W_attn=11.41e9, W_emb=2*129280*7168,
               W_dense_ffn=3*3*7168*18432, c_max=163_840, dense_mla=True,
               L_moe=58, kv_dense=512+64, P_act=37.2e9)
MODELS = [FLASH, PRO, R1]

# =====
# VALIDATION -- fail loudly if the constants drift
# =====
def _validate():
    for m, target in [(FLASH, 284e9), (PRO, 1572e9), (R1, 671e9)]:
        p = (m.L_moe*m.n_experts_per_layer*m.w_e + m.W_attn
              + m.W_dense_ffn + m.W_emb)
        assert abs(p/target - 1) < 0.01, f"{m.name}: derived {p/1e9:.1f}B vs
{target/1e9:.0f}B"
        assert abs(FLASH.kappa_hi - 3471) < 1 and abs(PRO.kappa_hi - 4970) < 1
        assert abs(R1.kappa_hi - 70272) < 1
_validate()

# =====
# TOPOLOGY -- a raw ceiling is not a deployable configuration
# =====
def snap(g, node=8):
    """Smallest valid parallelism degree >= g: powers of 2 up to a node,
    then whole nodes. Avoids returning e.g. 3 or 14 GPUs."""
    if g <= node:
        return next(x for x in (1, 2, 4, 8) if x >= g)
    return node * math.ceil(g / node)

# =====
# STAGES
# =====
def concurrency(N, dep, mix):
    """Stage 1: Little's Law. Returns [(name, C_i, c_i, lambda_i, hit_i)]."""
    out = []
    for name, s, c, o, r, hit in mix:
        lam = N * s * dep["a"] * r * dep["phi"] / 3600.0
        Wsvc = dep["T0"] + o * dep["tau"]
        out.append((name, lam*Wsvc/dep["load_factor"], c, lam, hit))
    return out

```



```

def size(m, N, dep, mix, hw):
    """Stages 2-4. Returns a dict; None if the model cannot serve the mix."""
    label, H, beta, flops, fp4_native = hw
    unmet = [n for n, _, c, _, _ in concurrency(N, dep, mix) if c > m.c_max]
    if unmet:
        return {"model": m.name, "hw": label, "infeasible": unmet}

    prec = "blackwell" if fp4_native else "hopper"
    Cs = concurrency(N, dep, mix)
    C = sum(x[1] for x in Cs)
    ctx_load = sum(Ci*c for _, Ci, c, _, _ in Cs) # sum C_i * c_i

    W = m.W(prec, dep["mtp"])
    M = W + dep["psi"] * m.kappa(dep["band"]) * ctx_load
    S = m.W_eff(C, prec, dep["mtp"], dep["skew"]) + m.A*C + m.B*ctx_load

    g_cap = M / (H * dep["eta_m"])
    g_bw = S / (beta * dep["eta_b"] * dep["tau"])

    # prefill: uncached prompt tokens/sec at peak, 2*P_act FLOPs/token
    pf_tps = sum(lam*c*(1-hit) for _, _, c, lam, hit in Cs)
    g_pf = pf_tps * 2 * m.P_act / (flops * dep["mfu"])

    g_dec = snap(math.ceil(max(g_cap, g_bw)))
    g_pf_s = snap(math.ceil(g_pf))
    bind = "bandwidth" if g_bw > g_cap else "capacity"
    return {"model": m.name, "hw": label, "prec": prec, "C": C, "W": W, "M": M,
            "S": S, "g_cap": g_cap, "g_bw": g_bw, "bind": bind, "pf_tps": pf_tps,
            "g_dec": g_dec, "g_pf": g_pf_s,
            "total": dep["R"]*(g_dec + g_pf_s)}

# =====
# DEFAULTS
# =====
DEPLOY = dict(
    a=0.45, # adoption: share of seats active per day
    phi=0.20, # peak-hour concentration
    psi=1.0, # idle KV multiplier (1.0 = evict at turn end)
    tau=0.040, # TPOT SL0, s/token
    T0=2.0, # TTFT budget, s
    eta_m=0.85, # usable HBM fraction
    eta_b=0.70, # achievable bandwidth fraction
    mfu=0.40, # prefill model-FLOPs utilisation
    skew=1.15, # expert routing skew on bandwidth (U4)
    load_factor=0.70, # don't plan past 70% of capacity (U7)
    band="hi", # KV accounting: "hi" conservative, "lo" paper-implied
    mtp=False, # load MTP module for speculative decoding (U3)
    R=2, # redundancy: N+1 / multi-AZ
)
# name share ctx out req/day prefix-cache hit

```

```

MIX = [("General chat",      0.60,   4_000,   600,   10,      0.30),
       ("Agentic coding",   0.25,  60_000,  2500,   40,      0.85),
       ("Doc analysis",     0.15, 250_000, 1500,    6,      0.20)]

# label, HBM bytes, HBM B/s, dense FP8 FLOP/s, native FP4?  <-- VERIFY FLOPS
HW = [("H100 80GB",      80*GB, 3.35*TB, 989e12, False),
      ("H200 141GB",   141*GB, 4.80*TB, 989e12, False),
      ("B200 192GB",   192*GB, 8.00*TB, 4500e12, True)]

# =====
def report(N, dep, mix, hw_list):
    print(f"{'model':10s}{m.W('blackwell')/GB:11.0f}{m.W('hopper')/GB:12.0f}"
          f"{'kappa lo-hi':>16s}{m.Phi():>7s}{m.Lambda('Mtok'):>15s}{m.C_sat():>7s}")
    for m in MODELS:
        print(f"{'m.name':10s}{m.W('blackwell')/GB:11.0f}{m.W('hopper')/GB:12.0f}"
              f"{'kappa lo-hi':>16s}{m.Phi():>7s}{m.Lambda('Mtok'):>15s}{m.C_sat():>7s}")

    Cs = concurrency(N, dep, mix)
    print(f"\nN={N:,}  ->  C={sum(x[1] for x in Cs):.0f} concurrent "
          f"(load factor {dep['load_factor']}), "
          f"mean ctx {sum(Ci*c for _,Ci,c,_ in Cs)/sum(x[1] for x in Cs):.0f}")

    for hw in hw_list:
        print(f"\n--- {hw[0]} ---")
        for m in MODELS:
            r = size(m, N, dep, mix, hw)
            if "infeasible" in r:
                print(f"  {m.name:10s} INFEASIBLE: context exceeds "
                      f"{m.c_max:,} for {'', '.join(r['infeasible'])}")
                continue
            print(f"  {r['model']:10s} [{r['prec']:9s}] "
                  f"M={r['M']/GB:6.0f}GB S={r['S']/GB:5.0f}GB/step "
                  f"| decode {r['g_dec']:3d} ({r['bind']}) "
                  f"+ prefill {r['g_pf']:3d} ({r['pf_tps']/1e3:.0f}K tok/s) "
                  f"| x[{dep['R']}] = {r['total']:3d} GPUs")

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--seats", type=int, default=5000)
    ap.add_argument("--band", choices=["lo", "hi"], default="hi")
    ap.add_argument("--mtp", action="store_true")
    ap.add_argument("--sweep", action="store_true")
    a = ap.parse_args()
    DEPLOY["band"], DEPLOY["mtp"] = a.band, a.mtp

    if a.sweep:
        print(f"{'seats':>8s}{m.C:>8s}" + "".join(f"{'m.name':>12s}" for m in MODELS)
              + " (B200, total incl. redundancy + prefill)")
        for n in (500, 1000, 2500, 5000, 10000, 25000, 50000):
            Cs = concurrency(n, DEPLOY, MIX)

```

```
    row = f"{n:8,d}{sum(x[1] for x in Cs):8.0f}"
    for m in MODELS:
        r = size(m, n, DEPLOY, MIX, HW[2])
        row += f"{'n/a' if 'infeasible' in r else r['total']:>12}"
    print(row)
else:
    report(a.seats, DEPLOY, MIX, HW)
```